Probability and Statistics for EECS Final Project Report

Ziheng Wang

August 19, 2023

1 Classical Bandit Algorithms

1.1 Expectation of the outcomes

Let $I(t) = I^* = \arg\max_{j \in \{1,2,3\}} \theta_j$ for $t = 1, 2, \ldots, 5000$, then

$$\max_{I(t), t=1,\ldots,5000} \mathbb{E}\left[\sum_{t=1}^{5000} r_{I(t)}\right] = 5000 \cdot \theta_{I^*} = 3500.$$

1.2 Implementation of the three algorithms

1.2.1 ϵ-greedy implementation

[ ]:
```
import matplotlib.pyplot as plt
import numpy as np
import random, math, copy

from tqdm import tqdm
from scipy.stats import beta
from skopt import gp_minimize, forest_minimize, gbrt_minimize
from skopt.plots import plot_convergence
```

[ ]:
```
def greedy(n, epsilon, seed):
    """
    params:
        n: total trials
        epsilon: value of epsilon
        seed: random seed
    return:
        total reward
    """
    num_actions = 3
    random.seed(seed)
    counts = np.zeros(num_actions)
    estimated_theta = np.zeros(num_actions)
    theta = np.array([0.7, 0.5, 0.4])
    total_rewards = 0
```

```
    for time_slot in range(n):
        # Exploration
        if np.random.random() < epsilon:
            action = np.random.randint(0, num_actions)
        # Exploitation
        else:
            action = np.argmax(estimated_theta)
        # Simulating the reward
        reward = 1 if np.random.random() < theta[action] else 0
        counts[action] += 1
        if reward == 1:
            total_rewards += 1
        # Update the estimated value of the action
        estimated_theta[action] += (reward - estimated_theta[action]) /␣
↪counts[action]

    return total_rewards
```

1.2.2 UCB implementation

[ ]:
```
def UCB(n, c, seed):
    """
    params:
        n: total trials
        c: value of UCB constant
        seed: random seed
    return:
        total reward
    """
    num_actions = 3
    random.seed(seed)
    theta = np.array([0.7, 0.5, 0.4])
    counts = np.zeros(num_actions)
    estimated_theta = np.zeros(num_actions)
    total_rewards = 0
    for time_slot in range(3):
        counts[time_slot] += 1
        estimated_theta[time_slot] = 1 if np.random.random() < theta[time_slot]␣
↪else 0

    for time_slot in range(3, n):
        calc = estimated_theta + c * np.sqrt(2 * np.log(time_slot + 1) / counts)
        action = np.argmax(calc)
        counts[action] += 1
        reward = 1 if np.random.random() < theta[action] else 0
        estimated_theta[action] += (1/counts[action]) * (reward -␣
↪estimated_theta[action])
```

```
        total_rewards += reward
    return total_rewards
```

1.2.3 TS implementation

[ ]:
```
def TS(n, a, b, seed):
    """
    params:
        n: total trials
        a: [a_1,a_2,a_3]
        b: [b_1,b_2,b_3]
        seed: random seed
    return:
        total reward
    """
    num_actions = 3
    np.random.seed(seed)
    theta = np.array([0.7, 0.5, 0.4])
    estimated_theta = np.zeros(num_actions)
    counts = np.zeros(num_actions)
    total_rewards = 0
    for time_slot in range(n):
        # Sample model
        for j in range(3):
            estimated_theta[j] = np.random.beta(a[j], b[j])

        #Select and pull the arm
        action = np.argmax(estimated_theta)
        counts[action] += 1

        # Update the distribution
        reward = 1 if np.random.random() < theta[action] else 0
        a[action] += reward
        b[action] += 1 - reward
        total_rewards += reward
    return total_rewards
```

1.3 Simulation over 200 trials

1.3.1 ϵ-greedy simulation with $\epsilon \in \{0.1, 0.5, 0.9\}$

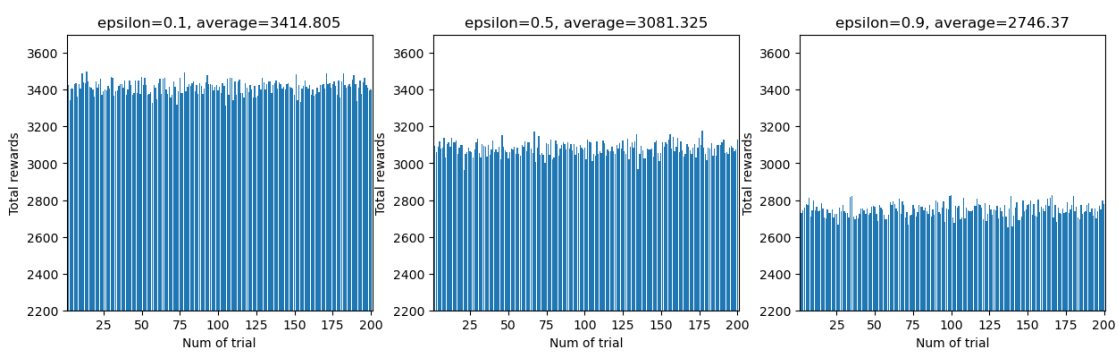
[ ]:
```
def repeat_greedy(trials=200):
    """
    return:
        List: the average reward when epsilon=0.1,0.5,0.9
        float: variance of the 200 trials' outcome
    """
```

```
    result_to_return = []
    var = []
    list_of_epsilon = [0.1, 0.5, 0.9]
    plt.figure(figsize=(15, 4))
    for i in range(len(list_of_epsilon)):
        result = np.zeros(trials)
        epsilon = list_of_epsilon[i]
        for trial in range(trials): # repeat 200 times
            result[trial] = greedy(5000, epsilon, trial) # reset the random seed
        # calculate the average outcome of this current epsilon
        average_reward = np.average(result)
        temp_var = np.var(result)
        plt.subplot(1, 3, i+1)
        plt.bar(np.arange(1, len(result)+1, 1), result)
        plt.xlim(1, len(result)+1); plt.ylim(2200, 3700)
        plt.xlabel("Num of trial"); plt.ylabel("Total rewards")
        plt.title("epsilon={}, average={}".format(epsilon, average_reward))
        print("Average rewards is {}, while average regret is {} for␣
↪epsilon={}".format(average_reward, np.abs(3500-average_reward), epsilon))
        result_to_return.append(average_reward)
        var.append(temp_var)
    plt.show()
    return result_to_return, var
```

Visulization

[ ]:
```
greedy_simulation_result, var_of_greedy = repeat_greedy()
```

```
Average rewards is 3414.805, while average regret is 85.19500000000016 for
epsilon=0.1
Average rewards is 3081.325, while average regret is 418.6750000000002 for
epsilon=0.5
Average rewards is 2746.37, while average regret is 753.6300000000001 for
epsilon=0.9
```



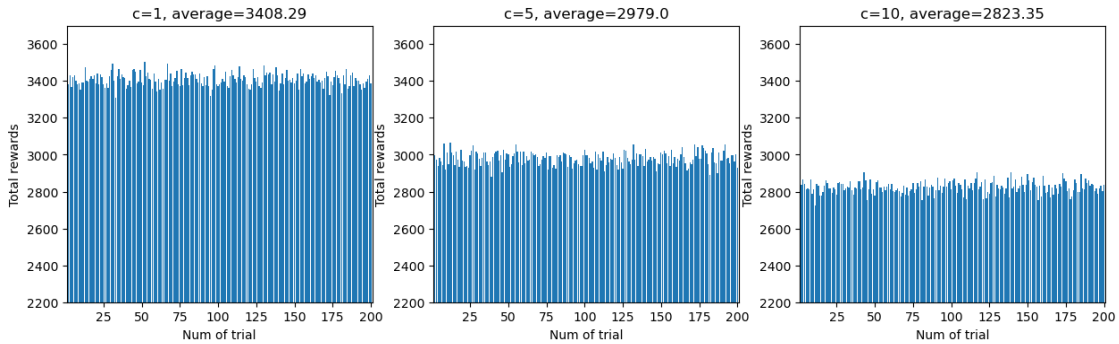
1.3.2 UCB simulation with $c \in \{1, 5, 10\}$

[ ]:
```
def repeat_ucb(trials=200):
    """
    return:
        List: the average reward when c=1,5,10
        float: variance of the 200 trials' outcome
    """
    result_to_return = []
    var = []
    list_of_constant = [1, 5, 10]
    plt.figure(figsize=(15, 4))
    for i in range(len(list_of_constant)):
        result = np.zeros(trials)
        constant = list_of_constant[i]
        for trial in range(trials): # repeat 200 times
            result[trial] = UCB(5000, constant, trial) # reset the random seed
        # calculate the average outcome of this current constant c
        average_reward = np.average(result)
        temp_var = np.var(result)
        plt.subplot(1, 3, i+1)
        plt.bar(np.arange(1, len(result)+1, 1), result)
        plt.xlim(1, len(result)+1); plt.ylim(2200, 3700)
        plt.xlabel("Num of trial"); plt.ylabel("Total rewards")
        plt.title("c={}, average={}".format(constant, average_reward))
        print("Average rewards is {}, while average regret is {} for c={}".
 ↪format(average_reward, np.abs(3500-average_reward), constant))
        result_to_return.append(average_reward)
        var.append(temp_var)
    plt.show()
    return result_to_return, var
```

Visualization

[ ]:
```
ucb_simulation_result, var_of_ucb = repeat_ucb()
```

```
Average rewards is 3408.29, while average regret is 91.71000000000004 for c=1
Average rewards is 2979.0, while average regret is 521.0 for c=5
Average rewards is 2823.35, while average regret is 676.6500000000001 for c=10
```



1.3.3 TS simulation with

- $\{(\alpha_1, \beta_1) = (1, 1), (\alpha_2, \beta_2) = (1, 1), (\alpha_3, \beta_3) = (1, 1)\}$
- $\{(\alpha_1, \beta_1) = (601, 401), (\alpha_2, \beta_2) = (401, 601), (\alpha_3, \beta_3) = (2, 3)\}$

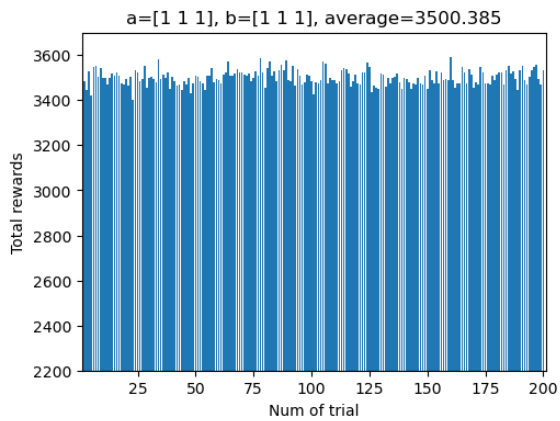
[ ]:
```
def repeat_ts(trials=200):
    """
    return:
        List: the average reward when a,b=([1,1,1], [1,1,1]), ([601, 401, 2],␣
↪[401, 601, 3])
        float: variance of the 200 trials' outcome
    """
    result_to_return = []
    var = []
    list_of_a = np.array([[1, 1, 1], [601, 401, 2]])
    list_of_b = np.array([[1, 1, 1], [401, 601, 3]])
    copy_of_a = copy.deepcopy(list_of_a)
    copy_of_b = copy.deepcopy(list_of_b)
    plt.figure(figsize=(12, 4))
    for i in range(2):
        result = np.zeros(trials)
        a = list_of_a[i]
        b = list_of_b[i]
        for trial in range(trials): # repeat 200 times
            result[trial] = TS(5000, a, b, trial) # reset the random seed
        # calculate the average outcome of this current a and b
        average_reward = np.average(result)
        temp_var = np.var(result)
        plt.subplot(1, 2, i+1)
        plt.bar(np.arange(1, len(result)+1, 1), result)
        plt.xlim(1, len(result)+1); plt.ylim(2200, 3700)
        plt.xlabel("Num of trial"); plt.ylabel("Total rewards")
        plt.title("a={}, b={}, average={}".format(copy_of_a[i], copy_of_b[i],␣
↪average_reward))
```

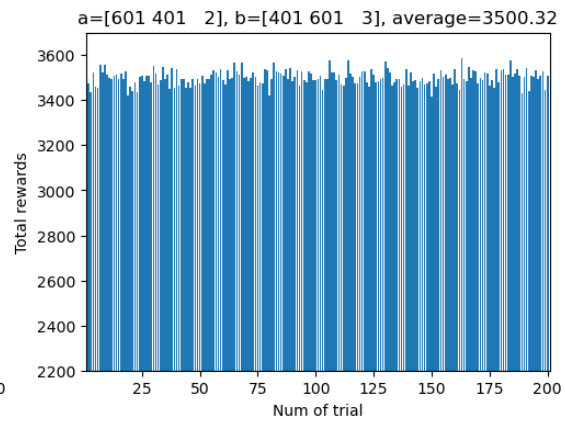
```
        print("Average rewards is {}, while average regret is {} for a={} and␣
↪b={}".format(average_reward, np.abs(3500-average_reward), copy_of_a[i],␣
↪copy_of_b[i]))
        result_to_return.append(average_reward)
        var.append(temp_var)
    plt.show()
    return result_to_return, var
```

Visualization

```
[ ]: ts_simulation_result, var_of_ts = repeat_ts()
```

```
Average rewards is 3500.385, while average regret is 0.3850000000002183 for a=[1
1 1] and b=[1 1 1]
Average rewards is 3500.32, while average regret is 0.3200000000001637 for
a=[601 401   2] and b=[401 601   3]
```





1.4 Comparison & Analysis of the algorithms

1.4.1 Performance Comparison

ϵ-greedy

Epsilon	Average reward	Variance	Gap
0.1	3413.72	1352.44	86.28
0.5	3081.11	1131.59	418.88
0.9	2746.43	1280.98	753.57

UCB

c	Average reward	Variance	Gap
1	3408.78	1160.82	91.21

c	Average reward	Variance	Gap
5	2980.27	1244.31	519.73
10	2824.01	1203.18	675.98

TS

prior beta distribution	Average reward	Variance	Gap
(1,1),(1,1),(1,1)	3500.38	1146.03	0.38
(601,401),(401,601),(2,3)	3500.32	1045.88	0.32

Based on the available data, the TS algorithm demonstrates superior performance in terms of accuracy and stability. It consistently achieves higher average rewards, exhibits a minimal gap compared to the oracle value, and demonstrates relatively low variances.

1.4.2 Impact of parameters

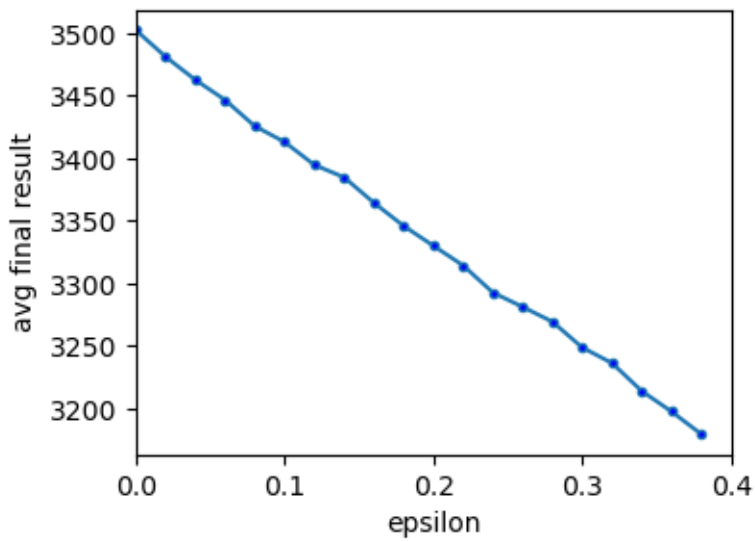
To assess the impact of parameters, we can conduct multiple iterations of the algorithm using different values of . By calculating the average rewards obtained in each case and plotting a corresponding curve, we can gain valuable insights into the relationship between parameter selection and algorithm performance.

ϵ-greedy

[ ]:
```
def epsilon_analyze(repeat=200):
    epsilon_list = np.arange(0, 0.02*20, 0.02)
    result = np.zeros_like(epsilon_list)
    for i in range(len(epsilon_list)):
        epsilon = epsilon_list[i]
        reward_list = np.zeros(repeat)
        for j in range(repeat):
            reward_list[j] = greedy(5000, epsilon, j)
        result[i] = np.average(reward_list)

    plt.figure(figsize=(4, 3))
    plt.plot(epsilon_list, result, marker='o', markerfacecolor='blue',␣
 ↪markersize=3)
    plt.xlim(0, 0.4)
    plt.xlabel("epsilon")
    plt.ylabel("avg final result")
    plt.show()

epsilon_analyze()
```



In the ϵ-greedy algorithm, the level of exploitation increases as the value of ϵ decreases. This means that with lower values of ϵ, the algorithm is more inclined to exploit the current estimations rather than explore other options.

According to our inference, the outcome when $\epsilon = 0$ is expected to be $5000 \times \theta_j$, in which j is the first arm that the player gets a reward from.

Contrary to initial expectations, the observed figure demonstrates a monotonic decrease in average rewards as ϵ increases. Surprisingly, in the absence of any exploration, the ϵ-greedy algorithm achieves the best performance, closely approaching the oracle value of 3500. This suggests that the player consistently selects the first arm in nearly every time slot, emphasizing exploitation over exploration.

The reason is probably as following, because of the `np.argmax()` function.

[ ]:
```
argmax_result = np.zeros(50)
zeros_vector = np.zeros(3)
for i in range(50):
    argmax_result[i] = np.argmax(zeros_vector)
print("Sum of 50 times argmax is {}".format(np.sum(argmax_result)))
```

```
Sum of 50 times argmax is 0.0
```

The result shows that `np.argmax()` returns the smallest index when there are multiple max value in an numpy array.

In the circumstance of ϵ-greedy algorithm, the three estimated θ value are initialized to 0, which makes the first `argmax()` return index 0, so the player will pull the first arm in the first time slot.

After that, whether he gets a reward or not, the `argmax()` will always return the index 0. And coincidentally, the first arm is the one of the largest θ. With the help of bias in UCB and TS algorithm, this problem will not appear anymore.

To avoid this, we can rewrite the argmax function to randomly choose an index if there are multiple max values in a numpy array containing 3 elements as follows:

[ ]:
```
def my_argmax(np_array):
    if np_array[0] == np_array[1] == np_array[2]:
        return np.random.randint(3)
    if np_array[0] == np_array[1] and np_array[0] > np_array[2]:
        return np.random.randint(2)
    if np_array[1] == np_array[2] and np_array[1] > np_array[0]:
        return np.random.randint(1, 3)
    if np_array[0] == np_array[2]:
        if np.random.random() < 0.5:
            return 0
        else:
            return 2
    return np.argmax(np_array)
```

[ ]:
```
def greedy_with_new_argmax(n, epsilon, seed):
    """
    params:
        n: total trials
        epsilon: value of epsilon
        seed: random seed
    return:
        total reward
    """
    num_actions = 3
    random.seed(seed)
    counts = np.zeros(num_actions)
    estimated_theta = np.zeros(num_actions)
    theta = np.array([0.7, 0.5, 0.4])
    total_rewards = 0

    for time_slot in range(n):
        # Exploration
        if np.random.random() < epsilon:
            action = np.random.randint(0, num_actions)
        # Exploitation
        else:
            action = my_argmax(estimated_theta)
        # Simulating the reward
        reward = 1 if np.random.random() < theta[action] else 0
        counts[action] += 1
        if reward == 1:
            total_rewards += 1
        # Update the estimated value of the action
```

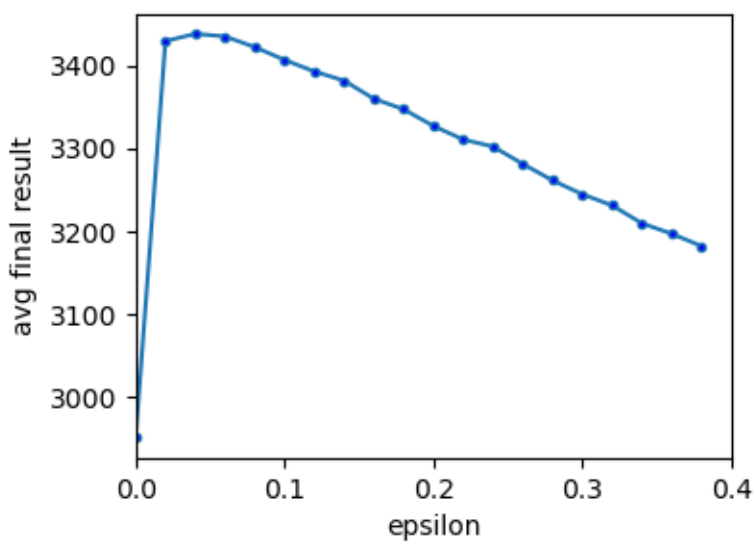
```
        estimated_theta[action] += (reward - estimated_theta[action]) /␣
↪counts[action]

    return total_rewards
```

[ ]:
```
def epsilon_analyze_with_new_argmax(repeat=200):
    epsilon_list = np.arange(0, 0.02*20, 0.02)
    result = np.zeros_like(epsilon_list)
    for i in range(len(epsilon_list)):
        epsilon = epsilon_list[i]
        reward_list = np.zeros(repeat)
        for j in range(repeat):
            reward_list[j] = greedy_with_new_argmax(5000, epsilon, j)
        result[i] = np.average(reward_list)

    plt.figure(figsize=(4, 3))
    plt.plot(epsilon_list, result, marker='o', markerfacecolor='blue',␣
↪markersize=3)
    plt.xlim(0, 0.4)
    plt.xlabel("epsilon")
    plt.ylabel("avg final result")
    plt.show()

epsilon_analyze_with_new_argmax()
```



After modifying the `argmax()` function, we observe that the resulting curve aligns with our intuition in the following ways:

1. When $\epsilon = 0$, the expectation of the total rewards given the arm index j (where the first

reward comes from) is $5000 \cdot \theta_j$. Since the arm chosen before receiving the first reward is random, it is most likely that the first arm chosen will have the largest θ value. By applying the Law of Total Expectation, we can deduce that

$$E(R) = \sum_{j \in \{1,2,3\}} E(R|j=i)P(j=i) \in (2500, 3500)$$

which matches the observed curve.

2. When ϵ is relatively small, its influence on determining whether the player randomly selects an arm becomes insignificant. The exploration towards the probability distribution of the arms is not substantial enough. Thus, it is reasonable to observe an interval between $\epsilon = 0$ and the point where total rewards are maximized, approximately around $\epsilon = 0.04$.
3. As ϵ increases, there is an excess of random arm selections once exploration has adequately informed the probability distribution. The curve tends to decline after $\epsilon = 0.04$ as the algorithm leans towards exploration even when a firm belief has been established about the probability distribution.

These observations support our intuition and highlight the relationship between the value of ϵ and the resulting behavior of the algorithm.

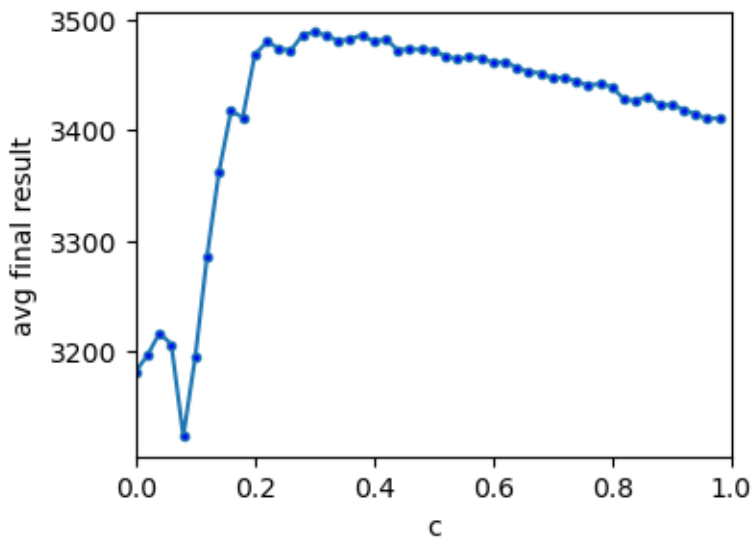
UCB

[ ]:
```
def ucb_analyze(repeat=200):
    c_list = np.arange(0, 1, 0.02)
    result = np.zeros_like(c_list)
    for i in tqdm(range(len(c_list))):
        c = c_list[i]
        reward_list = np.zeros(repeat)
        for j in range(repeat):
            reward_list[j] = UCB(5000, c, j)
        result[i] = np.average(reward_list)

    plt.figure(figsize=(4, 3))
    plt.plot(c_list, result, marker='o', markerfacecolor='blue', markersize=3)
    plt.xlim(0, 1)
    plt.xlabel("c")
    plt.ylabel("avg final result")
    plt.show()

ucb_analyze()
```

```
100%|      | 50/50 [03:10<00:00,  3.82s/it]
```



UCB algorithm is based on the unilateral Hoeffding Lemma[1], which we have proved in Homework 13 Problem 2.

$$P\left(\frac{1}{n}\sum_{i=1}^{n} X_i - \mu \leq -\epsilon\right) \leq exp\left(-\frac{2n\epsilon^2}{(b-a)^2}\right)$$

$$P\left(\mu \geq \frac{1}{n}\sum_{i=1}^{n} X_i + \epsilon\right) \leq exp\left(-\frac{2n\epsilon^2}{(b-a)^2}\right)$$

In bandit problem, $b - a = 1$, let $\epsilon = c \cdot U$, we have:

$$P\left(\theta \geq \hat{\theta} + c \cdot U\right) \leq e^{-2nc^2U^2}$$

i.e.,

$$P\left(\theta \leq \hat{\theta} + c \cdot U\right) \geq 1 - e^{-2nc^2U^2}$$

So $\hat{\theta} + c \cdot U$ becomes the Upper Confidence Bound.

Due to the fact:

$$U = \sqrt{\frac{2log(t)}{n}}$$

We can derive that the probability threshold is

$$1 - e^{-2nc^2U^2} = 1 - t^{-4c^2}$$

If $c = 1$, the right hand side becomes $\frac{15}{16}$ at the second time slot, which is already very close to 1. That makes the count value more determinant. Therefore, despite a well believed θ value we

have after several turns of pulling arms, the UCB algorithm under $c = 1$ may still waste some time pulling the arm with relatively low count despite its low estimated θ value.

If c is around 0.3, which seems to be the optimal value according to the simulation above, the speed of the threshold becoming tighter is lowered. For instance, when $t = 2$, the right hand side threshold becomes 22%, and 33% when $t = 3$. That makes the count value less dominant, being more reasonable for simulation and estimation of the real θ value.

TS The values

- $\{(\alpha_1, \beta_1) = (1, 1), (\alpha_2, \beta_2) = (1, 1), (\alpha_3, \beta_3) = (1, 1)\}$
- $\{(\alpha_1, \beta_1) = (601, 401), (\alpha_2, \beta_2) = (401, 601), (\alpha_3, \beta_3) = (2, 3)\}$

both provide a good enough simulation result as the average regert is less than 1.

Intuitively, the first setting is a weak prior distribution, with no additional knowledge. At the start of our experiment, the player pulls the arms and updates the beta parameters. Since that the prior distribution gave the player nothing, it will be super easy for him to change his belief towards the three arms. Therefore, the parameters will be approaching $\alpha_1/(\alpha_1 + \beta_1) = 0.7$, $\alpha_2/(\alpha_2 + \beta_2) = 0.5$, $\alpha_3/(\alpha_3 + \beta_3) = 0.4$ during the experiment process.

At the same time, the second setting gives the player a firm belief about the first and the second arm, that the first one has the biggest probability to produce reward. And this matches the truth. So, with the firm belief, the player will tend to pull the first arm always.

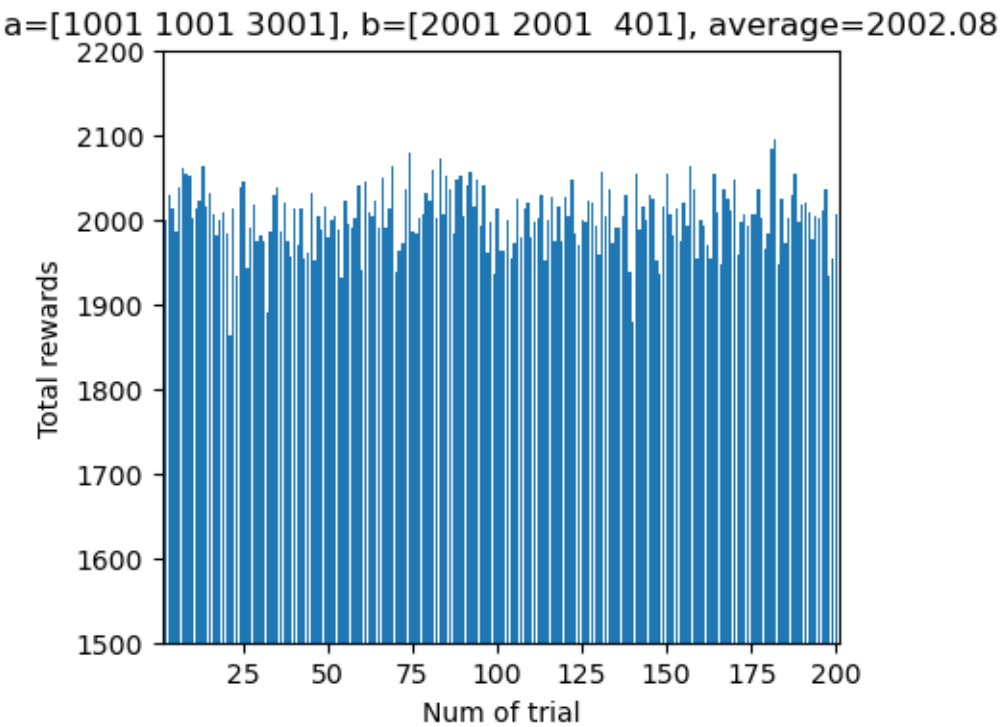
Therefore, if we provide a firm prior wrong information, for example, stating the third arm as the best one, we are supposed to get a bad result.

[ ]:
```
def repeat_ts_with_bad_prior_info(trials=200):
    result_to_return = []
    var = []
    list_of_a = np.array([[1001, 1001, 3001]])
    list_of_b = np.array([[2001, 2001, 401]])
    copy_of_a = copy.deepcopy(list_of_a)
    copy_of_b = copy.deepcopy(list_of_b)
    plt.figure(figsize=(10, 4))
    for i in range(1):
        result = np.zeros(trials)
        a = list_of_a[i]
        b = list_of_b[i]
        for trial in range(trials): # repeat 200 times
            result[trial] = TS(5000, a, b, trial) # reset the random seed
        # calculate the average outcome of this current a and b
        average_reward = np.average(result)
        temp_var = np.var(result)
        plt.subplot(1, 2, i+1)
        plt.bar(np.arange(1, len(result)+1, 1), result)
        plt.xlim(1, len(result)+1); plt.ylim(1500, 2200)
        plt.xlabel("Num of trial"); plt.ylabel("Total rewards")
```

```
        plt.title("a={}, b={}, average={}".format(copy_of_a[i], copy_of_b[i],␣
↪average_reward))
        print("Average rewards is {}, while average regret is {:1f} for a={}␣
↪and b={}".format(average_reward, np.abs(3500-average_reward), copy_of_a[i],␣
↪copy_of_b[i]))
        result_to_return.append(average_reward)
        var.append(temp_var)
    plt.show()

repeat_ts_with_bad_prior_info()
```

```
Average rewards is 2002.08, while average regret is 1497.920000 for a=[1001 1001
3001] and b=[2001 2001  401]
```



While it matches the expected θ value $0.4 \cdot 5000 = 2000$.

1.5 Exploration-exploitation trade-off

In the context of multi-arm bandit simulations, the primary objective is to maximize the cumulative reward by making a series of actions. Initially, when the algorithm possesses limited information about each arm, it is advisable to explore different arms in order to acquire knowledge about their respective reward distributions. This exploration phase enables the algorithm to estimate the expected rewards and variances associated with each arm.

As the algorithm accumulates more information through exploration, it can transition towards

exploitation by prioritizing the arms that have exhibited higher expected rewards. Exploitation involves selecting the arm that is believed to yield the highest reward based on the available knowledge. By focusing on exploiting the arm with the highest potential reward, the algorithm aims to optimize its overall rewards over time.

However, if the algorithm exclusively concentrates on exploitation, it may fail to discover potentially higher rewards from arms that have not been thoroughly explored. This is where the exploration component becomes crucial.

Thus, striking the right balance between exploration and exploitation is a pivotal consideration in bandit algorithms. This trade-off ensures a judicious allocation of resources between gathering information about unknown arms and maximizing potential rewards in decision-making scenarios with uncertain outcomes.

1.6 Dependent arms

1.6.1 Problem setting

To make the three arms be dependent on each other, we can give a prior knowledge that the sum of θ_1 and θ_2 is a constant. Given that, the first and the second arm are negatively correlated. Intuitively, the belief of getting rewards from second one would decrease if on a time slot we get a reward from pulling the first arm, and vice versa.

The detailed setting of this dependent arms problem is:

- $N = 5000$.
- $\theta_1 = 0.7, \theta_2 = 0.5, \theta_3 = 0.4$ but unknown to the algorithm.
- Given that $\theta_1 + \theta_2 = c$, in which $c \in \mathbb{R}^+$.

The pseudocode describing the modified-UCB algorithm is:

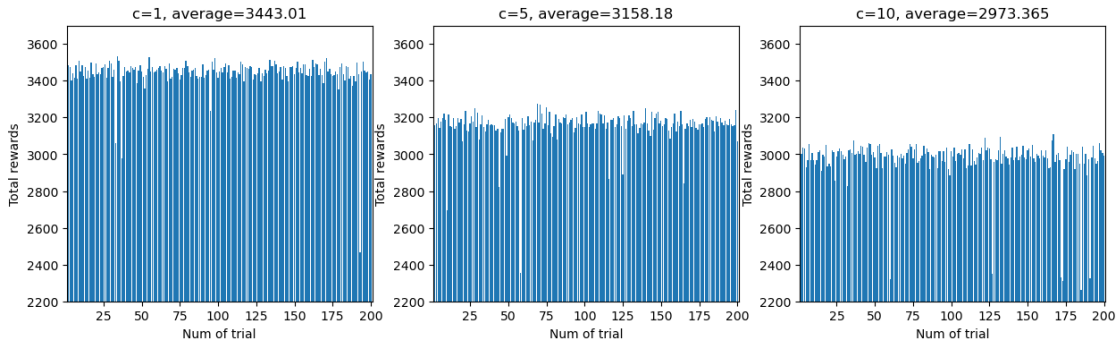
[ ]:
```
def modified_UCB(n, c, seed):
    """
    params:
        n: total trials
        c: value of UCB constant
        seed: random seed
    return:
        total reward
    """
    num_actions = 3
    random.seed(seed)
    theta = np.array([0.7, 0.5, 0.4])
    counts = np.zeros(num_actions)
    estimated_theta = np.zeros(num_actions)
    total_rewards = 0
    for time_slot in range(3):
        counts[time_slot] += 1
        estimated_theta[time_slot] = 1 if np.random.random() < theta[time_slot]␣
↪else 0
    for time_slot in range(3, n):
```

```
        calc = estimated_theta + c * np.sqrt(2 * np.log(time_slot + 1) / counts)
        action = np.argmax(calc)
        counts[action] += 1
        reward = 1 if np.random.random() < theta[action] else 0
        estimated_theta[action] += (1/counts[action]) * (reward -␣
↪estimated_theta[action])
        if action == 0:
            counts[1] += 1
            estimated_theta[1] += (1/counts[1]) * (0 - estimated_theta[1]) if␣
↪reward == 1 else (1/counts[1]) * (1 - estimated_theta[1])
        elif action == 1:
            counts[0] += 1
            estimated_theta[0] += (1/counts[0]) * (0 - estimated_theta[0]) if␣
↪reward == 1 else (1/counts[0]) * (1 - estimated_theta[0])
        total_rewards += reward
    return total_rewards
```

[ ]:
```
def repeat_modified_ucb(trials=200):
    """
    return:
        List: the average reward when c=1,5,10
        float: variance of the 200 trials' outcome
    """
    result_to_return = []
    var = []
    list_of_constant = [1, 5, 10]
    plt.figure(figsize=(15, 4))
    for i in range(len(list_of_constant)):
        result = np.zeros(trials)
        constant = list_of_constant[i]
        for trial in range(trials): # repeat 200 times
            result[trial] = modified_UCB(5000, constant, trial) # reset the␣
↪random seed
        # calculate the average outcome of this current constant c
        average_reward = np.average(result)
        temp_var = np.var(result)
        plt.subplot(1, 3, i+1)
        plt.bar(np.arange(1, len(result)+1, 1), result)
        plt.xlim(1, len(result)+1); plt.ylim(2200, 3700)
        plt.xlabel("Num of trial"); plt.ylabel("Total rewards")
        plt.title("c={}, average={}".format(constant, average_reward))
        print("Average rewards is {}, while average regret is {} for c={}".
↪format(average_reward, np.abs(3500-average_reward), constant))
        result_to_return.append(average_reward)
        var.append(temp_var)
    plt.show()
    return result_to_return, var
```

[ ]:
```
modified_UCB_simulation_result, var_of_modified_ucb = repeat_modified_ucb()
```

```
Average rewards is 3443.01, while average regret is 56.98999999999978 for c=1
Average rewards is 3158.18, while average regret is 341.82000000000016 for c=5
Average rewards is 2973.365, while average regret is 526.6350000000002 for c=10
```



Comparing the result with UCB without any prior knowledge of dependency, the average rewards have increased for approximately $36, 185, 158$ for $c = 1, 5, 10$, respectively, which is a effectively increase.

2 Bayesian Bandit Algorithms

2.1 Simulation of intuitive policy

[ ]:
```
success_prob = [0.4, 0.6]
def bayes_bandit(n, gamma=1):
    """
    params:
        n: total trials
        prob: probability of each bandit arm
    return:
        total reward, chosen arm
    """
    num_actions = 2
    theta = np.array([0.4, 0.6])
    beta_param_1 = np.array([1, 1])
    beta_param_2 = np.array([1, 1])
    counts = np.zeros(num_actions)
    total_rewards = 0
    for time_slot in range(n):
        expactation_1 = beta_param_1[0] / (beta_param_1[0] + beta_param_1[1])
        expactation_2 = beta_param_2[0] / (beta_param_2[0] + beta_param_2[1])
        if expactation_1 > expactation_2:
            action = 0
        if expactation_1 < expactation_2:
```

```
            action = 1
        if expactation_1 == expactation_2:
            action = np.random.randint(2)
        reward = (gamma ** time_slot) * (1 if np.random.random() <␣
↪theta[action] else 0)
        counts[action] += 1
        if action == 0:
            beta_param_1[0] += reward
            beta_param_1[1] += 1 - reward
        else:
            beta_param_2[0] += reward
            beta_param_2[1] += 1 - reward
        total_rewards += reward

    expactation_1 = beta_param_1[0] / (beta_param_1[0] + beta_param_1[1])
    expactation_2 = beta_param_2[0] / (beta_param_2[0] + beta_param_2[1])

    return total_rewards

print("Total rewards when gamma = 1 is {}".format(bayes_bandit(5000)))
```

```
Total rewards when gamma = 1 is 3033
```

If we change the gamma value ranging from 0.5 to 0.9, we can then evaluate the accuacy by calculating the average simulation error.

[ ]:
```
bayes_bandit_naive_result = []
for i in range(5, 10):
    gamma = 0.1 * i
    sum = np.zeros(1000)
    for j in range(1000):
        sum[j] = bayes_bandit(5000, gamma=gamma)
    expected_reward = 0.6 / (1-gamma)
    bayes_bandit_naive_result.append(np.abs(np.average(sum) - expected_reward) /
↪ expected_reward)
```

[ ]:
```
bayes_bandit_naive_result = np.array(bayes_bandit_naive_result)
print("The average error of 5 different gamma values is {:.2f}%.".format(np.
↪average(bayes_bandit_naive_result) * 100))
```

```
The average error of 5 different gamma values is 14.79%.
```

2.2 Example of its disadvantage

As analyzed in the Problem 1.3, when the prior distribution gives wrong but firm information about the probability distribution about the arms, we will end up to pulling the wrong arm all the time.

2.3 Proof

1. $R(\alpha_1, \beta_1, \alpha_2, \beta_2) = \max\{R_1(\alpha_1, \beta_1), R_2(\alpha_2, \beta_2)\}$ holds because expectedly, the optimal policy will be pulling the arm with higher θ all the time through the play.
2. For $R_1(\alpha_1, \beta_1) = \frac{\alpha_1}{\alpha_1+\beta_1}[1 + \gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)] + \frac{\beta_1}{\alpha_1+\beta_1}[\gamma R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2)]$:
   - The term $\frac{\alpha_1}{\alpha_1+\beta_1}$ represents the probability that the first turn is a success. Conditioning on this, the first turn gives back a reward with value 1. To deal with the rest of the turns, since that at each time slot, the reward value is γ^{t-1}. If the prior beta distribution is given by $(\alpha_1 + 1, \beta_1)$, the maximized rewards is $R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)$, we can consider the rest of the turns as delaying the process in $R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)$ for a time slot, and the total rewards would be $\gamma R(\alpha_1 + 1, \beta_1, \alpha_2, \beta_2)$.
   - The term $\frac{\beta_1}{\alpha_1+\beta_1}$ represents the probability that the first turn is a failure. Conditioning on this, the first turn gives back no rewards. For the rest of the turns, we can consider similarly as a delayed process of $R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2)$, thus the maximized rewards would be $\gamma R(\alpha_1, \beta_1 + 1, \alpha_2, \beta_2)$.
3. Similarly, we can derive that $R_2(\alpha_2, \beta_2) = \frac{\alpha_2}{\alpha_2+\beta_2}[1 + \gamma R(\alpha_1, \beta_1, \alpha_2 + 1, \beta_2)] + \frac{\beta_2}{\alpha_2+\beta_2}[\gamma R(\alpha_1, \beta_1, \alpha_2, \beta_2 + 1)]$.